

Class Notes: Array Operations in Data Structures

1. Array

An **array** is a linear data structure that stores a fixed number of elements of the **same data type** in contiguous memory locations.

For example:

Index:	0	1	2	3	4
Array:	10	20	30	40	50

Here, **10** is stored at index **0**, **20** at index **1**, and so on.

Basic Operations on Arrays

The major operations are:

1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Updating
6. Sorting
7. Merging

2. Traversal of an Array

Traversal means visiting each element of an array exactly once, generally from the first element to the last.

For example:

A = [10, 20, 30, 40, 50]

Traversal:

10 → 20 → 30 → 40 → 50

Algorithm: Array Traversal

TRAVERSE(A, N)

1. Start
2. Set $i = 0$
3. Repeat while $i < N$
 - Print $A[i]$
 - $i = i + 1$
4. Stop

C Program

```
C

#include <stdio.h>

int main()
{
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Array elements are: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input/Output

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

Array elements are: 10 20 30 40 50
```

Complexity Analysis

All n elements must be visited.

Therefore,

Time Complexity = $O(n)$

No additional array is required.

Auxiliary Space = $O(1)$

3. Insertion in an Array

Insertion is the operation of adding a new element at a specified position in an array.

Suppose:

`A = [10, 20, 30, 40, 50]`

Insert `25` at position `3`.

Before insertion:

`10 20 30 40 50`

Move elements toward the right:

`10 20 _ 30 40 50`

Insert `25` :

`10 20 25 30 40 50`

Algorithm: Insertion

Here, `POS` represents the position starting from **1**.

```
INSERT(A, N, POS, ITEM)
```

1. Start
2. If `POS < 1` or `POS > N + 1`
 Print "Invalid Position"
 Stop
3. Set `i = N - 1`
4. Repeat while `i >= POS - 1`
 `A[i + 1] = A[i]`
 `i = i - 1`
5. Set `A[POS - 1] = ITEM`
6. Set `N = N + 1`
7. Stop

C Program: Insertion

C

```
#include <stdio.h>

int main()
{
    int a[100], n, i, pos, item;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position for insertion: ");
    scanf("%d", &pos);

    printf("Enter element to insert: ");
    scanf("%d", &item);

    if(pos < 1 || pos > n + 1)
    {
        printf("Invalid position");
        return 0;
    }

    for(i = n - 1; i >= pos - 1; i--)
        a[i + 1] = a[i];

    a[pos - 1] = item;
    n++;

    printf("Array after insertion: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input/Output

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

Enter position for insertion: 3
```

Enter element to insert: 25

Array after insertion: 10 20 25 30 40 50

4. Complexity Analysis of Insertion

The complexity depends on where the new element is inserted.

Case 1: Insertion at End

```
10 20 30 40 50
           ↓
           60
10 20 30 40 50 60
```

No existing element needs to be shifted.

Time Complexity = $O(1)$

Case 2: Insertion at Beginning

Before:

```
10 20 30 40 50
```

Insert 5 :

```
5 10 20 30 40 50
```

Every existing element has to be shifted one position to the right.

Number of shifts $\approx n$

Therefore,

Time Complexity = $O(n)$

Case 3: Insertion at Middle

Approximately $n/2$ elements may need to be shifted.

Therefore:

Time Complexity = $O(n)$

because constants such as $1/2$ are ignored in asymptotic analysis.

Summary

Insertion	Complexity
At beginning	$O(n)$
At middle	$O(n)$
At end	$O(1)$
Worst case	$O(n)$

Auxiliary Space = $O(1)$

5. Deletion from an Array

Deletion means removing an existing element from an array.

Suppose:

```
A = [10, 20, 30, 40, 50]
```

Delete the element at position **3**.

Element **30** is removed:

```
10 20 __ 40 50
```

Shift subsequent elements toward the left:

```
10 20 40 50
```

Algorithm: Deletion by Position

```
DELETE(A, N, POS)
```

```
1. Start
2. If POS < 1 or POS > N
    Print "Invalid Position"
    Stop
3. Set ITEM = A[POS - 1]
4. Set i = POS - 1
5. Repeat while i < N - 1
    A[i] = A[i + 1]
    i = i + 1
6. Set N = N - 1
7. Print deleted ITEM
8. Stop
```

6. C Program: Deletion by Position

C

```
#include <stdio.h>

int main()
{
    int a[100], n, i, pos, item;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position to delete: ");
    scanf("%d", &pos);

    if(pos < 1 || pos > n)
    {
        printf("Invalid position");
        return 0;
    }

    item = a[pos - 1];

    for(i = pos - 1; i < n - 1; i++)
        a[i] = a[i + 1];

    n--;

    printf("Deleted element: %d\n", item);

    printf("Array after deletion: ");
```

```
for(i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
return 0;  
}
```

Input/Output

```
Enter number of elements: 5  
Enter 5 elements: 10 20 30 40 50  
  
Enter position to delete: 3  
  
Deleted element: 30  
Array after deletion: 10 20 40 50
```

7. Complexity Analysis of Deletion

Deletion from Beginning

```
10 20 30 40 50  
  
Delete 10  
  
20 30 40 50
```

All remaining elements must be shifted.

Time Complexity = $O(n)$

Deletion from Middle

Some elements have to be shifted left.

Time Complexity = $O(n)$

Deletion from End

```
10 20 30 40 50  
      ↓  
      Delete  
  
10 20 30 40
```

No shifting is required.

Time Complexity = $O(1)$

Summary

Deletion	Complexity
Beginning	$O(n)$
Middle	$O(n)$
End	$O(1)$
Worst case	$O(n)$

Auxiliary Space = $O(1)$

8. Searching in an Array

Searching means finding whether a particular element exists in an array and, if it exists, determining its location.

For an unsorted array, **Linear Search** can be used.

Example:

```
A = [10, 20, 30, 40, 50]
```

```
Search = 40
```

Comparisons:

```
10 ≠ 40
```

```
20 ≠ 40
```

```
30 ≠ 40
```

```
40 = 40 ✓
```

Therefore, 40 is found at position 4.

Algorithm: Linear Search

```
LINEAR_SEARCH(A, N, ITEM)
```

```
1. Start
```

```
2. Set i = 0
3. Repeat while i < N
    If A[i] == ITEM
        Print "Found at position", i + 1
        Stop
    i = i + 1
4. Print "Element not found"
5. Stop
```

9. C Program: Linear Search

```
C

#include <stdio.h>

int main()
{
    int a[100], n, i, item, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &item);

    for(i = 0; i < n; i++)
    {
        if(a[i] == item)
        {
            printf("Element found at position %d", i + 1);
            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Element not found");

    return 0;
}
```

Input/Output

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

Enter element to search: 40

Element found at position 4
```

Complexity

Best case: element is first.

$O(1)$

Average case: approximately $n/2$ comparisons.

$O(n)$

Worst case: element is last or not present.

$O(n)$

10. Updating an Array Element

Updating means changing the value stored at a particular array position.

Example:

```
A = [10, 20, 30, 40, 50]
```

Change position 3 to 35 .

```
Before: 10 20 30 40 50
```

↓

```
After: 10 20 35 40 50
```

Because an array supports direct access, the required location can be accessed immediately.

Algorithm

UPDATE(A, N, POS, ITEM)

1. Start
2. If POS < 1 or POS > N
 Print "Invalid Position"
 Stop
3. Set A[POS - 1] = ITEM
4. Stop

C Program

```
C

#include <stdio.h>

int main()
{
    int a[100], n, i, pos, item;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter position to update: ");
    scanf("%d", &pos);

    printf("Enter new value: ");
    scanf("%d", &item);

    if(pos < 1 || pos > n)
    {
        printf("Invalid position");
        return 0;
    }

    a[pos - 1] = item;

    printf("Array after update: ");

    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Input/Output

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

Enter position to update: 3
Enter new value: 35

Array after update: 10 20 35 40 50
```

Complexity

If the position/index is known:

Time Complexity = $O(1)$

Auxiliary Space = $O(1)$

If we first have to **search for the value** and then update it, the overall complexity can become **$O(n)$** .

11. Accessing an Array Element

An important operation of arrays is **random/direct access**.

For example:

```
A = [10, 20, 30, 40, 50]

A[0] = 10
A[2] = 30
A[4] = 50
```

The address of an element can be calculated directly from its index.

For a zero-based one-dimensional array:

```
Address(A[i]) =
Base_Address + i × Size_of_Element
```

Therefore, accessing any array element takes constant time.

Time Complexity = $O(1)$

12. Deletion by Value

Sometimes the value to delete is given instead of its position.

Example:

```
A = [10, 20, 30, 40, 50]
```

```
Delete = 30
```

Two operations are required:

```
Search for 30
```

```
↓
```

```
Find position
```

```
↓
```

```
Delete 30
```

```
↓
```

```
Shift remaining elements
```

Result:

```
10 20 40 50
```

Algorithm

```
DELETE_VALUE(A, N, ITEM)
```

1. Start
2. Set POS = -1
3. For i = 0 to N - 1
 If A[i] == ITEM
 POS = i
 Break
4. If POS == -1
 Print "Element not found"
 Stop
5. For i = POS to N - 2
 A[i] = A[i + 1]
6. N = N - 1
7. Stop

Complexity Analysis

Searching:

$O(n)$

Shifting:

$O(n)$

Hence,

$$O(n) + O(n)$$

Constants are ignored.

Therefore:

Overall Time Complexity = $O(n)$

not $O(n^2)$.

13. Complete C Program for Array Operations

The following program combines the major array operations into one menu-driven program.

C

```
#include <stdio.h>

int main()
{
    int a[100];
    int n, i, choice;
    int pos, item, found;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    do
    {
        printf("\n\n--- ARRAY OPERATIONS ---");
        printf("\n1. Traversal");
        printf("\n2. Insertion");
        printf("\n3. Deletion");
        printf("\n4. Searching");
        printf("\n5. Update");
```

```
printf("\n6. Exit");

printf("\nEnter your choice: ");
scanf("%d", &choice);

switch(choice)
{
    case 1:
        printf("Array elements: ");

        for(i = 0; i < n; i++)
            printf("%d ", a[i]);

        break;

    case 2:
        printf("Enter position: ");
        scanf("%d", &pos);

        printf("Enter element: ");
        scanf("%d", &item);

        if(pos < 1 || pos > n + 1)
        {
            printf("Invalid position");
            break;
        }

        for(i = n - 1; i >= pos - 1; i--)
            a[i + 1] = a[i];

        a[pos - 1] = item;
        n++;

        printf("Element inserted successfully.");
        break;

    case 3:
        printf("Enter position to delete: ");
        scanf("%d", &pos);

        if(pos < 1 || pos > n)
        {
            printf("Invalid position");
            break;
        }

        item = a[pos - 1];

        for(i = pos - 1; i < n - 1; i++)
            a[i] = a[i + 1];
```

```

        n--;

        printf("%d deleted successfully.", item);
        break;

    case 4:
        printf("Enter element to search: ");
        scanf("%d", &item);

        found = 0;

        for(i = 0; i < n; i++)
        {
            if(a[i] == item)
            {
                printf("Element found at position %d", i + 1);
                found = 1;
                break;
            }
        }

        if(found == 0)
            printf("Element not found.");

        break;

    case 5:
        printf("Enter position to update: ");
        scanf("%d", &pos);

        if(pos < 1 || pos > n)
        {
            printf("Invalid position");
            break;
        }

        printf("Enter new value: ");
        scanf("%d", &item);

        a[pos - 1] = item;

        printf("Element updated successfully.");
        break;

    case 6:
        printf("Program terminated.");
        break;

    default:
        printf("Invalid choice.");
}

```

```
} while(choice != 6);  
  
    return 0;  
}
```

Sample Execution

```
Enter number of elements: 5  
Enter 5 elements: 10 20 30 40 50
```

```
--- ARRAY OPERATIONS ---
```

1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Update
6. Exit

```
Enter your choice: 2
```

```
Enter position: 3  
Enter element: 25
```

```
Element inserted successfully.
```

```
--- ARRAY OPERATIONS ---
```

1. Traversal
2. Insertion
3. Deletion
4. Searching
5. Update
6. Exit

```
Enter your choice: 1
```

```
Array elements: 10 20 25 30 40 50
```

14. Complete Complexity Analysis of Array Operations

Let n be the number of elements in the array.

Operation	Best Case	Average Case	Worst Case	Auxiliary Space
Access by index	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Traversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$

Operation	Best Case	Average Case	Worst Case	Auxiliary Space
Update by index	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Insert at beginning	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Insert at middle	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Insert at end*	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Delete from beginning	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete from middle	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete from end	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Delete by value	$O(1)**$	$O(n)$	$O(n)$	$O(1)$

* Assuming sufficient unused capacity exists in the array.

** Best case can be $O(1)$ if the value is found at the last position and no shifting is required; in general, deletion by value involves both searching and possible shifting.

15. Why Array Insertion and Deletion are $O(n)$

This is an important examination question.

Arrays store elements in **contiguous memory locations**.

Consider:

Index	0	1	2	3	4
	10	20	30	40	50

To insert 15 at position 2:

```

Step 1:  10   20   30   40   50
           →   →
           →   →

Step 2:  10   —   20   30   40   50

Step 3:  10   15   20   30   40   50

```

Many elements have to be moved.

Similarly, deletion creates an empty position that must be filled by shifting elements toward the left.

Hence, both insertion and deletion may require shifting up to n elements.

Therefore:

```

Insertion =  $O(n)$ 
Deletion  =  $O(n)$ 

```

in the worst case.

16. Important Points for Examination

Array characteristics:

- Elements are stored in contiguous memory locations.
- All elements normally have the same data type.
- Array indexing in C starts from 0 .
- Direct access is very efficient: **$O(1)$** .
- Traversal requires visiting all elements: **$O(n)$** .
- Searching an unsorted array using Linear Search: **$O(n)$** .
- Insertion at the beginning/middle requires shifting: **$O(n)$** .
- Deletion from the beginning/middle requires shifting: **$O(n)$** .
- Insertion/deletion at the end can be **$O(1)$** when no shifting or resizing is required.
- Updating an element when its index is known: **$O(1)$** .
- Arrays are efficient for **random access**, but comparatively inefficient for frequent **insertion and deletion in the middle**.